

TREE 1

PROBLEM 2

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 62
Question description There are three common ways to traverse the nodes of a binary tree: • <i>Preorder</i>: First process the root, then the left subtree, and finally the right subtree. • <i>Inorder</i>: First process the left subtree, then the root, and finally the right subtree. • <i>Postorder</i>: First process the left subtree, then the right subtree, and finally the root. There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal. Constraints • $1 \leq n \leq 10^5$ Input The first input line has an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$. After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers. You can assume that the input corresponds to a binary tree. Output Print the postorder traversal of the tree.					

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 5 3 2 1 4 3 5 1 2 4 EXPECTED OUTPUT 3 1 4 2 5	INPUT (STDIN) 6 6 5 3 2 1 4 6 3 5 1 2 4 EXPECTED OUTPUT 3 1 4 2 5 6

Mandatory Test Cases

Test Case 1
KEYWORD for(i=1;i<=n;i++)

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 8	TOKEN COUNT 390	NLOC 41

SOURCE CODE:

```
#include <stdio.h>

int locate_index(int arr[], int target, int left, int right) {
    for (int idx = left; idx <= right; idx++) {
        if (arr[idx] == target) return idx;
    }
    return -1;
}

void build_postorder(int pre[], int in[], int in_start, int in_end, int *pre_ptr) {
    if (in_start > in_end) return;

    int current_val = pre[(*pre_ptr)++];
    int pivot = locate_index(in, current_val, in_start, in_end);

    build_postorder(pre, in, in_start, pivot - 1, pre_ptr);
    build_postorder(pre, in, pivot + 1, in_end, pre_ptr);

    printf("%d ", current_val);
}

int main() {
    int node_count;
    if (scanf("%d", &node_count) != 1) return 0;

    int preorder_seq[node_count];
    int inorder_seq[node_count];
    for (int i = 0; i < node_count; i++) scanf("%d", &preorder_seq[i]);
    for (int i = 0; i < node_count; i++) scanf("%d", &inorder_seq[i]);

    int pre_idx = 0;
    build_postorder(preorder_seq, inorder_seq, 0, node_count - 1, &pre_idx);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
5
5 3 2 1 4
3 5 1 2 4
3 1 4 2 5
```

```
6
6 5 3 2 1 4
6 3 5 1 2 4
3 1 4 2 5 6
```